.NET
Aspire Microservices

A Practical Guide to Interview Preparation

Basic Concepts

1. Microservices Architecture and how does it differ from monolithic architecture?

Answer: Microservices architecture is an approach where an application is composed of small, independent services that communicate over a network. Each service owns a specific capability and can be developed, deployed, and scaled independently.

- Monolithic Architecture: Single unified codebase; tightly coupled; easier initially but harder to scale and maintain.
- Microservices Architecture: Multiple loosely coupled services; independent deployment; easier to scale and evolve.

  Example: A monolith might handle Auth + Catalog + Orders in one codebase. In microservices, you split them into Auth Service, Catalog Service, Orders Service.

2. Benefits of using microservices

Answer: Microservices help teams ship faster and scale components independently, but they require good engineering discipline.

- Scalability: scale only the hot services.
- Independent deployment: release without redeploying the whole system.
- Fault isolation: one service failing does not necessarily crash everything.
- Technology flexibility: services can use the best fit tech.

3. Main challenges when working with microservices

Answer: Microservices introduce complexity in development and operations because you are building a distributed system.

- Complexity: many moving parts and environments.
- Data consistency: cross-service transactions are hard.
- Inter-service communication: networking failures, latency, retries.
- Testing: end-to-end testing is more complex.
- Monitoring: requires strong observability (logs, metrics, traces).

4. How to implement microservices in .NET?

Answer: In .NET, microservices are commonly built with ASP.NET Core (REST/minimal APIs), gRPC for service-to-service calls, and messaging (Kafka/RabbitMQ/Azure Service Bus) for async workflows.

- ASP.NET Core: production-ready HTTP services.
- gRPC: high-performance contract-based RPC.
- Background services (Workers): event handlers, schedulers, outbox processors.

```
// Minimal API example
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/hello", () => "Hello, World!");
app.Run();
```

5. What is .NET Aspire and why is it used?

Answer: .NET Aspire is a set of tools and libraries to build, run, and observe distributed .NET apps. It provides code-first orchestration, service discovery, and a local dashboard for telemetry.

- AppHost: orchestrates projects and resources.
- ServiceDefaults: shared observability and health defaults.
- Dashboard: view logs, traces, and metrics.

6. How does Aspire simplify microservices development?

Answer: Aspire reduces local setup friction by orchestrating services and their dependencies (databases, Redis, etc.) and wiring connection strings automatically.

- No manual port juggling.
- No custom scripts to start dependencies.
- Built-in OpenTelemetry setup.

```
// AppHost: add a service project
builder.AddProject<Projects.CatalogApi>("catalog");
```

Advanced Concepts

7. How to handle service discovery in .NET microservices?

Answer: Service discovery helps microservices locate each other dynamically. With Aspire, services can be addressed by logical names, and Aspire wires the addresses at runtime.

- Aspire service discovery: use logical names.
- In Kubernetes: use DNS service names.
- With Dapr: sidecar name-based invocation.

  Example: call http://catalog instead of hardcoding localhost:5001.

8. How do microservices communicate with each other?

Answer: Communication can be synchronous or asynchronous. Choose based on coupling, latency, and consistency needs.

- Synchronous: HTTP/REST, gRPC.
- Asynchronous: message brokers and event streams.
- Async reduces coupling and improves resilience.

9. What is an API Gateway and why do we use it?

Answer: An API Gateway is a reverse proxy that routes client requests to the right services and handles cross-cutting concerns.

- Authentication/authorization
- Rate limiting
- Request routing and aggregation
- Observability and logging

10. How do you implement an API Gateway in .NET?

Answer: YARP (Yet Another Reverse Proxy) is a modern option to build an API gateway in .NET.

- Dynamic routing
- Load balancing
- Transforms (headers/path/body)
- Extensible middleware pipeline

```
// YARP basic config snippet (appsettings.json)
{
  "ReverseProxy": {
    "Routes": {
      "route1": {
        "ClusterId": "catalogCluster",
        "Match": { "Path": "/catalog/{**catch-all}" }
      }
    },
    "Clusters": {
      "catalogCluster": {
        "Destinations": { "d1": { "Address": "http://catalog" } }
      }
    }
  }
}
```

11. What are retries, timeouts, and circuit breakers?

Answer: They are resilience patterns to handle transient failures and prevent cascading outages in distributed systems.

- Retries: re-attempt transient failures with backoff.
- Timeouts: cap how long calls can wait.
- Circuit breakers: stop calling unhealthy dependencies temporarily.

12. How do you implement Circuit Breaker in .NET?

Answer: Use Polly (or built-in .NET resilience integrations for HttpClient) to apply policies consistently.

- Circuit breaker for repeated failures
- Retry with jitter and backoff
- Timeout policy

```
// Polly example
var policy = Policy
  .Handle<HttpRequestException>()
  .CircuitBreakerAsync(3, TimeSpan.FromSeconds(30));
```

13. How do you manage configuration in distributed .NET systems?

Answer: Use externalized configuration and secure secret stores instead of hardcoding config values.

- ASP.NET Core configuration providers (JSON, env vars, Key Vault).
- Kubernetes ConfigMaps/Secrets.
- Centralized config (e.g., Azure App Configuration).

14. How do you handle authentication and authorization (OAuth2/JWT)?

Answer: Use OpenID Connect/OAuth2 with JWT access tokens. Validate tokens at the gateway or service boundary.

- Use JWT bearer authentication.
- Prefer short-lived access tokens.
- Use scopes/roles for authorization.

```
// JWT bearer auth (Program.cs)
builder.Services.AddAuthentication("Bearer")
  .AddJwtBearer("Bearer", options => {
      options.Authority = "https://issuer.example";
      options.Audience  = "api";
  });
```

15. What is the role of health checks in microservices?

Answer: Health checks help orchestrators and monitoring tools determine if services are alive and ready.

- Liveness: process is running.
- Readiness: dependencies are healthy.
- Used by Kubernetes probes and dashboards.

Design and Implementation

16. How do you design a microservices architecture for a complex application?

Answer: Start with domain boundaries (DDD), define clear APIs, and enforce data ownership per service.

- Identify bounded contexts.
- Define APIs and contracts.
- Decouple via events when possible.
- Choose a data consistency strategy.
- Plan for observability and resilience.

17. Best practices for designing RESTful APIs in microservices?

Answer: Design stable, discoverable APIs with versioning and consistent error handling.

- Use nouns for resources: /users, /orders.
- Use proper HTTP verbs and status codes.
- Stateless requests.
- Pagination for collections.
- Standardize error formats (ProblemDetails).

```
// API versioning using URL segment
app.MapGet("/v1/users", () => Results.Ok(/* ... */));
```

18. How do you implement API versioning in ASP.NET Core?

Answer: Use a clear versioning strategy (URI, header, or media type) and a deprecation policy.

- URI versioning: /v1/...
- Header versioning: api-version: 1.0
- Document versioning and sunset old versions.

19. How do you handle data consistency in microservices?

Answer: Avoid distributed transactions when possible; prefer eventual consistency and patterns like Saga and Outbox.

- Saga pattern: coordinated local transactions with compensations.
- Outbox pattern: transactional event publishing.
- Event-driven workflows for consistency.

20. What strategies exist for transactions across multiple services?

Answer: Distributed transactions (2PC) are complex and risky; sagas are generally preferred.

- Two-phase commit (rare; expensive).
- Saga orchestration/choreography.
- Event sourcing for audit and rebuild.

21. Role of Docker in deploying .NET microservices?

Answer: Docker packages each microservice with its dependencies into a container for consistent runtime across environments.

- Reproducible builds
- Isolation

- Easy scaling and orchestration with Kubernetes

```
# Dockerfile example (.NET)
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY . .
ENTRYPOINT ["dotnet", "CatalogApi.dll"]
```

22. Describe the process of deploying microservices using Docker and Kubernetes

Answer: Build the container image, push to a registry, and deploy with Kubernetes manifests or Helm.

- Dockerize each service.
- Create Deployment + Service YAML.
- Apply manifests (kubectl apply).
- Configure autoscaling and observability.

```
# kubectl apply
kubectl apply -f deployment.yaml
```

23. How do you scale microservices?

Answer: Scale horizontally (more instances) and use autoscaling based on CPU/requests/custom metrics.

- Horizontal scaling: replicas.
- Vertical scaling: more CPU/memory.
- Autoscaling: HPA/KEDA.

24. What is the sidecar pattern and how is it used?

Answer: A sidecar is a helper container deployed next to a service to handle cross-cutting concerns.

- Service mesh proxy (Envoy/Istio).
- Dapr sidecar for service invocation/state/pub-sub.
- Log/metrics collectors.

25. How do you manage configuration changes without redeploying?

Answer: Use centralized configuration stores and hot-reload patterns when safe.

- Azure App Configuration with refresh.
- Kubernetes ConfigMap reload (with restart or sidecar).
- Feature flags for controlled rollouts.

Testing and Debugging

26. How do you test microservices in isolation?

Answer: Test each service with unit tests and mocked dependencies; use contract tests for integration boundaries.

- Unit tests for domain logic.
- Mock external dependencies.
- Contract testing (Pact).

27. How do you handle inter-service communication failures?

Answer: Use timeouts, retries with backoff, circuit breakers, and fallbacks.

- Retries with exponential backoff.
- Circuit breakers to stop cascades.
- Fallback responses when appropriate.
- Timeouts to avoid resource exhaustion.

28. What is Domain-Driven Design (DDD) and why is it relevant?

Answer: DDD helps define bounded contexts that map naturally to microservices and reduce coupling.

- Bounded contexts
- Ubiquitous language
- Clear ownership of models and data

29. What are tools and techniques for end-to-end testing?

Answer: Use integration test environments plus API tests and UI tests if needed.

- Testcontainers for dependencies.
- API tests (Postman/newman).
- Playwright for UI if required.
- Service virtualization for external APIs.

30. How do you perform integration testing in .NET microservices?

Answer: Run the service with its dependencies (often via Testcontainers) and test through HTTP/gRPC endpoints.

- Use WebApplicationFactory for ASP.NET Core.
- Use ephemeral DB containers for clean tests.
- Use mocks for external APIs.

```
// WebApplicationFactory example
public class ApiTests : IClassFixture<WebApplicationFactory<Program>>
{
  // ...
}
```

31. Best practices for exception handling in ASP.NET Core microservices?

Answer: Centralize error handling, standardize error responses, and avoid leaking internals.

- Use ProblemDetails (RFC7807).
- Global exception handling middleware.
- Consistent error codes.

Logging, Monitoring, and Observability

32. How do you handle logging in microservices?

Answer: Use structured logging and centralize logs for querying and correlation.

- Serilog structured logs.
- Centralize with ELK/OpenSearch.
- Correlate with trace/span IDs.

```
// Serilog basic setup
Log.Logger = new LoggerConfiguration()
  .Enrich.FromLogContext()
  .WriteTo.Console()
  .CreateLogger();
```

33. What is distributed tracing and why does it matter?

Answer: Tracing tracks a request across multiple services to identify latency and failures.

- OpenTelemetry
- Jaeger/Zipkin backends
- Trace context propagation

34. How do you monitor the health of services?

Answer: Expose health endpoints and integrate with dashboards and alerting.

- ASP.NET Core HealthChecks.
- Kubernetes probes.
- Aspire dashboard for local dev.

```
// Health checks
builder.Services.AddHealthChecks();
app.MapHealthChecks("/health");
```

35. Explain OpenTelemetry usage in .NET microservices

Answer: OpenTelemetry provides a standard way to collect metrics, logs, and traces. Many .NET libraries integrate directly.

- Instrument ASP.NET Core, HttpClient, SQL clients.
- Export to an OTLP collector.
- Visualize in Grafana/Jaeger.

36. How does Aspire help with observability?

Answer: Aspire provides defaults and a dashboard that surfaces logs, metrics, traces, and service status during development.

- Automatic wiring of telemetry.
- Central dashboard view.
- Faster debugging locally.

CI/CD, Deployment, and Production

37. What is CI/CD and how is it applied to microservices?

Answer: CI builds/tests each service; CD deploys services independently with safe rollout strategies.

- CI: build + test + scan.
- CD: deploy to staging/prod.
- Independent pipelines per service.

38. What are blue-green deployments?

Answer: Two environments (blue=old, green=new). Switch traffic after validation; easy rollback.

- Zero downtime switching
- Fast rollback

39. How do you implement canary releases?

Answer: Gradually shift traffic to the new version while monitoring metrics and errors.

- Service mesh or ingress traffic splitting.
- Feature flags for user targeting.
- Automated rollback on SLO breach.

40. How do you ensure high availability?

Answer: Design for redundancy, statelessness, and failure recovery.

- Multiple replicas
- Load balancing
- Failover
- Health checks + auto-restart

41. How do you handle secret management?

Answer: Use dedicated secret stores; never commit secrets to source control.

- Azure Key Vault / AWS Secrets Manager / HashiCorp Vault
- Kubernetes Secrets (encrypted at rest)
- Rotate secrets regularly

Real-World Scenarios and Anti-Patterns

42. Describe a scenario where you successfully implemented microservices

Answer: Example: E-commerce platform with separate services for Users, Catalog, Orders, Payments and an event-driven backbone.

- Technologies: ASP.NET Core, gRPC, Kafka/Azure Service Bus, Docker, Kubernetes
- Challenges: data consistency, communication failures, monitoring
- Solutions: Saga/Outbox, Polly resilience, OpenTelemetry + dashboards

43. How do you ensure backward compatibility during changes?

Answer: Avoid breaking changes; version APIs and use consumer-driven contract tests.

- API versioning and deprecation policy
- Contract testing
- Additive changes over breaking changes

44. Common microservices anti-patterns and how to avoid them

Answer: Avoid architectural mistakes that increase coupling and operational pain.

- Shared database schema across services (avoid: each service owns its DB).
- Chatty synchronous calls (avoid: async events / aggregation).
- Too many tiny services (avoid: meaningful bounded contexts).
- Tight coupling via shared libraries/models (avoid: separate contracts).

45. How do you implement caching in microservices?

Answer: Caching improves latency and reduces load. Use distributed caches (Redis) and cache invalidation strategies.

- Cache-aside pattern
- TTL-based expiration
- Invalidate on writes / publish invalidation events

```
// Caching (sketch)
public async Task<User> GetUser(long id)
{
  // Try cache -> fallback DB -> set cache
  return default!;
}
```

46. Role of message brokers in async communication

Answer: Message brokers decouple services, improve resilience, and enable event-driven workflows.

- Kafka: event streaming
- RabbitMQ: queues and routing
- Azure Service Bus: enterprise messaging

47. How do you validate data in .NET services?

Answer: Use validation at API boundaries and domain invariants in core logic.

- FluentValidation or DataAnnotations
- ProblemDetails for validation errors
- Reject invalid input early

```
public record CreateUserRequest(
  [System.ComponentModel.DataAnnotations.Required]
  string Name
);
```

48. What tools and frameworks have you used for building and managing .NET microservices?

Answer: Common tools include ASP.NET Core, gRPC, Docker, Kubernetes, OpenTelemetry, YARP, Dapr, and .NET Aspire for local orchestration.

- Building: ASP.NET Core, Worker Services, gRPC
- Managing: Docker, Kubernetes, Helm, GitHub Actions/Azure DevOps
- Observability: OpenTelemetry + Grafana/Jaeger
- Gateway: YARP

49. How do you handle dependency injection in .NET microservices?

Answer: ASP.NET Core has built-in DI. Register services in the container and inject via constructors for testability.

- Prefer constructor injection.
- Use scoped lifetime for request services.
- Avoid ServiceLocator anti-pattern.

```
// DI registration
builder.Services.AddScoped<IUserRepo, UserRepo>();
```

50. How do you handle load balancing in microservices?

Answer: Load balancing can be handled at multiple layers: gateway, infrastructure, or service mesh.

- Kubernetes Services/Ingress
- Gateway (YARP) load balancing
- Service mesh (Istio/Linkerd)

51. What is the Outbox pattern?

Answer: Outbox ensures events are published reliably by writing events to an outbox table within the same DB transaction as the business change.

- Prevents lost events
- Works well with eventual consistency
- Usually paired with a background publisher

52. What is the Saga pattern?

Answer: Saga manages distributed workflows via a sequence of local transactions with compensation on failure.

- Choreography (events)
- Orchestration (central coordinator)
- Prefer idempotent handlers

53. What is idempotency and why is it important?

Answer: Idempotency means repeating the same request produces the same result. It protects systems from retries and duplicate messages.

- Use idempotency keys

- Deduplicate message processing
- Design PUT/DELETE as idempotent

54. How do you implement rate limiting in .NET APIs?

Answer: Use ASP.NET Core rate limiting middleware or enforce at the gateway/ingress.

- Per-IP limits
- Token bucket/leaky bucket algorithms
- Apply to expensive endpoints

55. How do you do distributed caching safely?

Answer: Use TTL, cache invalidation strategies, and avoid caching sensitive data unintentionally.

- TTL + eviction policy
- Invalidate on writes
- Cache stampede prevention (locks/jitter)

56. How do you migrate a monolith to microservices?

Answer: Use incremental extraction (Strangler Fig). Start with low-risk bounded contexts and introduce a gateway.

- Identify seams
- Extract service + DB ownership
- Use events for integration
- Measure and iterate

57. What is the role of CQRS in microservices?

Answer: CQRS separates read models from write models to optimize performance and simplify scaling.

- Read model optimized for queries
- Write model enforces invariants
- Often used with event sourcing

58. How do you prevent cascading failures?

Answer: Use timeouts, bulkheads, circuit breakers, and backpressure.

- Polly policies
- Queue-based async communication
- Resource limits in Kubernetes

59. How do you approach observability in production?

Answer: Define SLIs/SLOs, centralize telemetry, and create actionable alerts.

- Golden signals: latency, traffic, errors, saturation
- Trace sampling strategy
- Alert on symptoms not noise

60. What's the difference between readiness and liveness probes?

Answer: Readiness indicates if the service can receive traffic; liveness indicates if the process is alive and should be restarted.

- Readiness: dependency checks
- Liveness: basic process health

- Avoid heavy checks in liveness

61. How do you secure service-to-service communication?

Answer: Use mTLS or service identity, plus least-privilege authorization.

- Service mesh mTLS
- JWT with audience/scopes
- Rotate certs/keys

62. How do you handle schema evolution in event-driven systems?

Answer: Use schema versioning and backward-compatible changes for events.

- Add fields (backward compatible)
- Avoid renaming/removing fields abruptly
- Use schema registry where possible

63. How do you handle retries for message processing?

Answer: Use delayed retries and dead-letter queues to prevent poison messages from blocking processing.

- Exponential backoff
- DLQ for manual inspection
- Idempotent consumers

64. What is a Dead Letter Queue (DLQ)?

Answer: A DLQ stores messages that could not be processed successfully after retries.

- Protects main queue
- Enables debugging
- Avoid silent drops